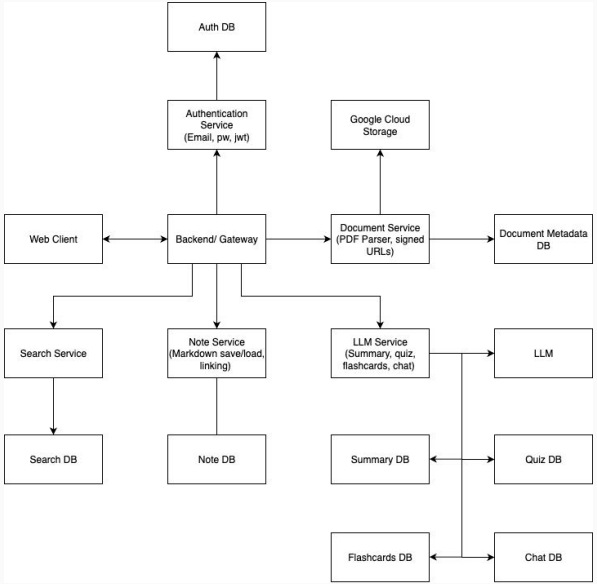
Edunote
Midpoint Check-in

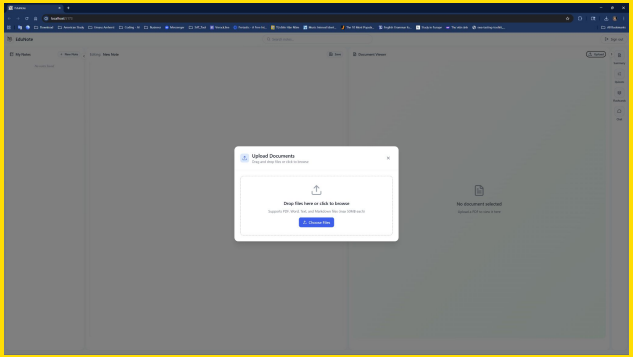
Kiet Chu, Beatrice Calvelo, Bao Dao, Anh Tran

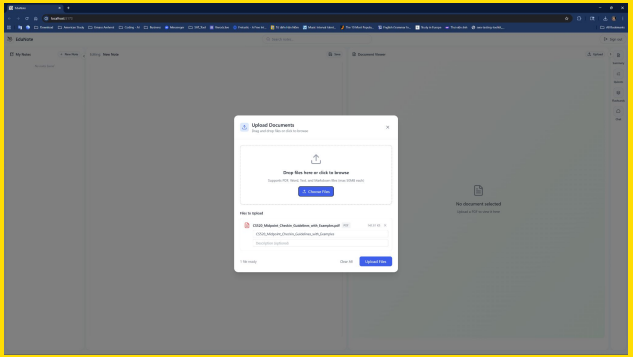
Project Summary ↘

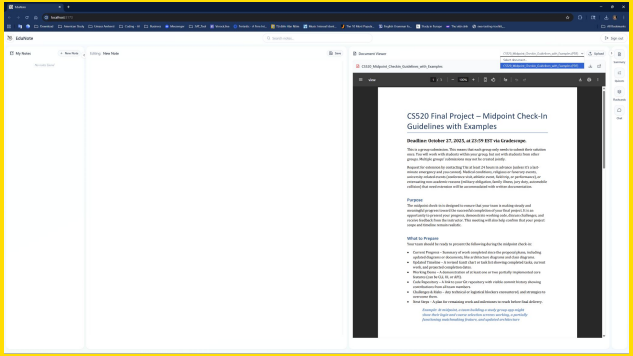


EduNote is an AI-powered smart notebook that merges PDF-anchored note-taking with on-demand tutoring features-summaries, quiz/flashcard generation, and chat-inside one interface. Target users are students who currently jump between notes, search engines, and Q&A sites; EduNote streamlines this flow by keeping assistance context-aware and attached to each note.

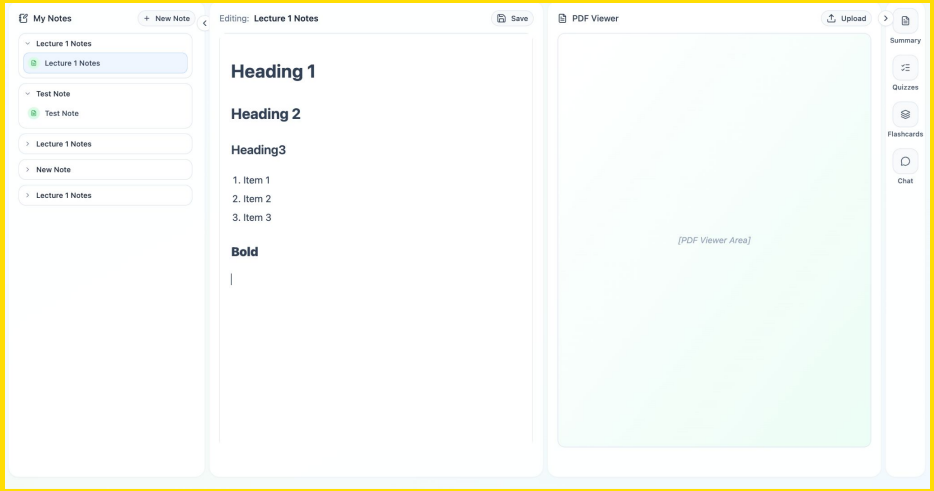
Anh Tran: Upload and view pdf files

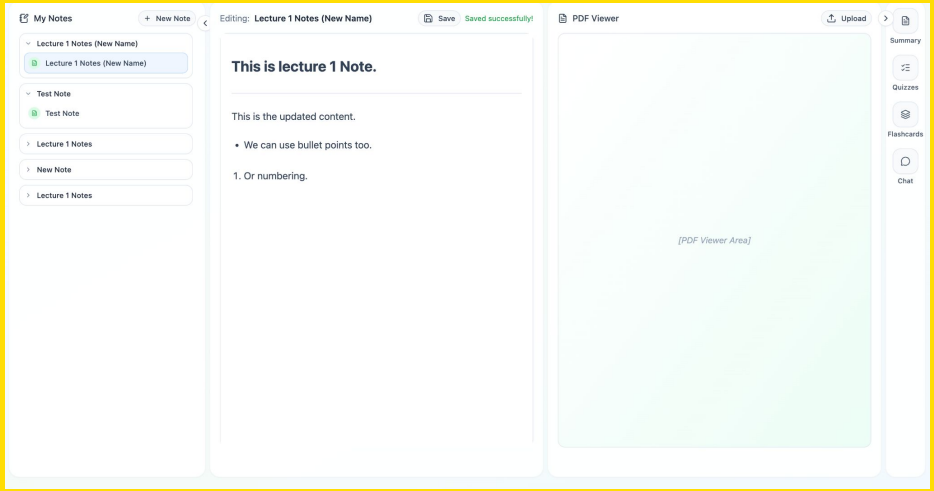




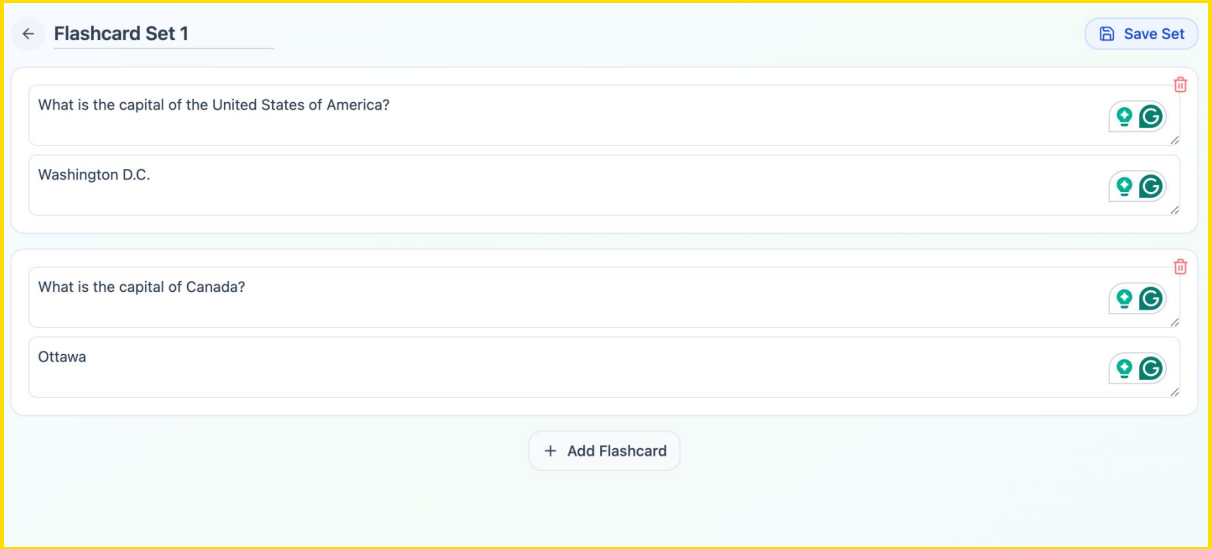


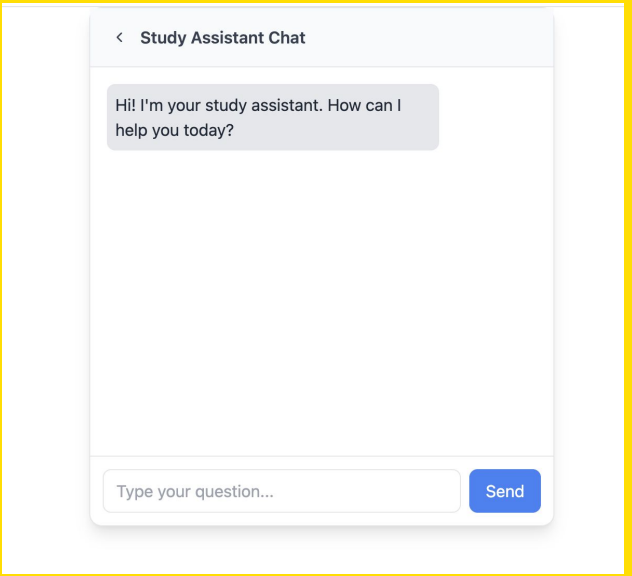
Kiet Chu: Create and Update Notes



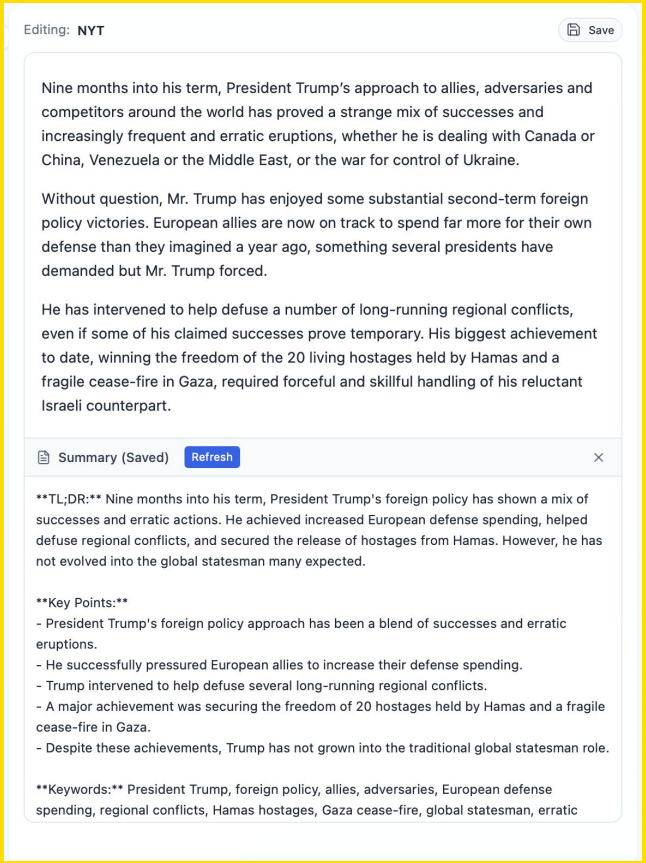


Beatrice Calvelo: Flashcard and Chat UIs





Kevin: AI Summary



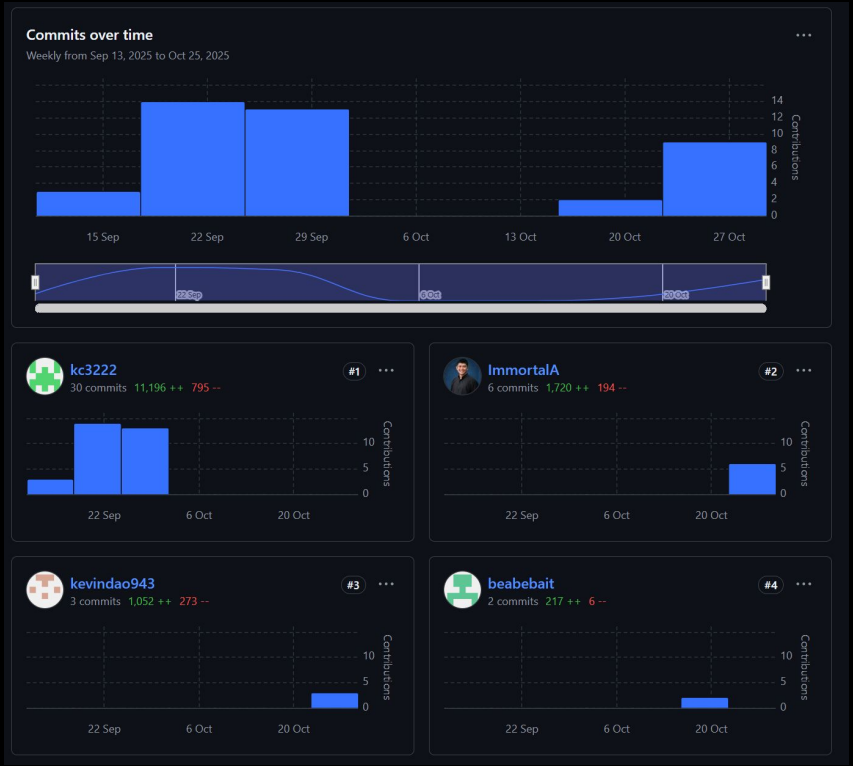
When clicking the Summary button on the right panel
→ Call gemini-2.5-flash to summarize the text
Automatically save the generation for next time
'Refresh' button to regenerate the summary
'Close' button to close the summarization

Updated Timeline

Window	Milestone	Status
Week 1–3	Repo/cloud setup, DB schema, base React/FastAPI	Completed
Week 3–5	Notes CRUD, PDF upload + viewer, rich editor	Completed
Week 5–6	Auth backend + session/token guards	In progress (priority)
Week 6–7	PDF parser service (chunking + metadata)	In progress
Week 7–8	Quiz/Flashcard service+DB + UIs	In progress
Week 8–9	Summary service+DB + Summary UI	Next
Week 9–10	Chat service+DB (wire to UI)	Planned
Week 10–11	OpenAI rate-limit handling, retries, logs	Planned
Week 11–12	Polish, tests, bug-fix, accessibility	Planned
Week 13	Final demo & docs	Planned

Code Repository & Commit History

https://github.com/kc3222/EduNote





Challenges/Risks & Mitigation Plans

Top challenges: imports/packaging in microservices; DB schema drift; deps/env consistency; external APIs (OpenAI/Gemini); service orchestration.

Key risks: leaked keys; import/runtime errors; schema mismatches/data loss; missing packages/broken AI calls; CORS/auth gaps; startup race conditions.

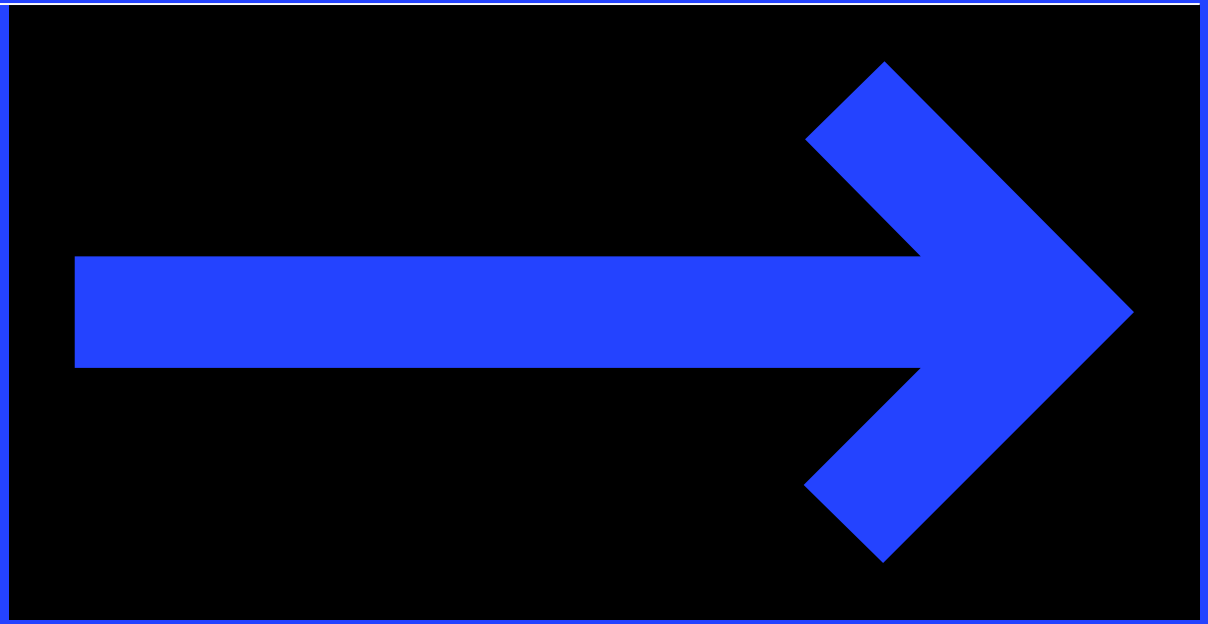
Mitigations (high-level): `__init__.py` + absolute imports + `python -m`; Alembic migrations w/ versioning & rollback; pinned `requirements*.txt` + `environment.yml`; env vars + timeouts/retries/mock mode; docker-compose/Makefile with healthchecks; consistent CORS allowlist + JWT.

Security & secrets: never commit keys; use `.env` + `.env.example` + `.gitignore`; secret scanning (pre-commit/GitHub); key rotation; least-privilege tokens.

Reliability & UX: graceful degradation (cache/mock LLM outputs), exponential backoff, user-friendly errors; structured logs + basic metrics (5xx rate, LLM latency).

Q&A

Any questions?
Ask away!



Thank you!